

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester IV)
AI(Artificial Intelligence)
Practical VII Part 4 & 5

Roll NO :- T053	Name :- Nidhi Yadav
Calss :- TYIT	Batch :- 02
Date of Assignment :- 07-08-2026	Date of Submission :- 07-08-2026

Aim:- Machine Learning

4. Implement k-fold cross-validation on a classification model.

Code:-

```
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target
# Create classification model
model = KNeighborsClassifier()
# Apply 5-fold cross-validation
scores = cross_val_score(model, X, y, cv=5)
print("Accuracy scores for each fold:")
print(scores)
print("Average Accuracy:", round(scores.mean() * 100, 2), "%")
```

Output:-

```
Accuracy scores for each fold:
[0.96666667 1.          0.93333333 0.96666667 1.
Average Accuracy: 97.33 %
```

5. Implement a Decision Tree classifier using the Iris or any relevant dataset.

Code:-

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
# Create model
model = DecisionTreeClassifier()
# Train model
model.fit(X_train, y_train)
# Display accuracy
print("Training Accuracy:", model.score(X_train, y_train) * 100)
print("Testing Accuracy:", model.score(X_test, y_test) * 100)
# Prediction
prediction = model.predict([X_test[0]])
print("Predicted Class:", prediction[0])
```

Output:-

```
Training Accuracy: 100.0
Testing Accuracy: 95.55555555555556
Predicted Class: 0
```

Dataset:-zomato.csv**Code:-**

```
import pandas as pd
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
# Load dataset
data = pd.read_csv("zomato.csv")
# Remove missing values
data = data[['votes', 'listed_in(type)']].dropna()
# Convert target into numeric values
data['listed_in(type)'] = data['listed_in(type)'].astype('category').cat.codes
# Input and Output
X = data[['votes']]
y = data['listed_in(type)']
# Create classification model
model = KNeighborsClassifier()
# Apply 5-fold cross validation
scores = cross_val_score(model, X, y, cv=5)
print("Accuracy scores for each fold:")
print(scores)
print("Average Accuracy:", round(scores.mean() * 100, 2), "%")
```

Output:-

```
Accuracy scores for each fold:
[0.47399459 0.48588554 0.48254858 0.47510394 0.48921976]
Average Accuracy: 48.14 %
```

Code:-

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
# Load dataset
data = pd.read_csv("zomato.csv")
# Remove missing values
data = data[['votes', 'listed_in(type)']].dropna()
# Convert target into numeric values
data['listed_in(type)'] = data['listed_in(type)'].astype('category').cat.codes
# Input and Output
X = data[['votes']]
y = data['listed_in(type)']
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
# Create model
model = DecisionTreeClassifier()
# Train model
model.fit(X_train, y_train)
# Display accuracy
print("Training Accuracy:", model.score(X_train, y_train) * 100)
print("Testing Accuracy:", model.score(X_test, y_test) * 100)
# Prediction
prediction = model.predict([X_test.iloc[0]])
print("Predicted Class:", prediction[0])
```

Output:-

```
Training Accuracy: 53.13389132896881
Testing Accuracy: 48.16963134828564
Predicted Class: 2
```

Basis	Iris Dataset (KNN)	Iris Dataset (Decision Tree)	Zomato Dataset (KNN)	Zomato Dataset (Decision Tree)
Dataset	Built-in Iris dataset	Built-in Iris dataset	Zomato CSV dataset	Zomato CSV dataset
Algorithm	K-Nearest Neighbors	Decision Tree	K-Nearest Neighbors	Decision Tree
Data Loading	load_iris()	load_iris()	pd.read_csv()	pd.read_csv()
Data Preprocessing	Not required	Not required	Missing values removed and target converted into numbers	Missing values removed and target converted into numbers
Validation Method	5-Fold Cross Validation	Train-Test Split	5-Fold Cross Validation	Train-Test Split
Model Training	Done automatically by cross_val_score()	Using model.fit()	Done automatically by cross_val_score()	Using model.fit()
Accuracy	Average accuracy of all folds	Training and Testing accuracy	Average accuracy of all folds	Training and Testing accuracy
Prediction	No	Yes	No	Yes

Conclusion

- KNN is useful for checking the overall performance of the model using Cross Validation.
- Decision Tree is useful for training the model and predicting the class of new data.
- The Iris dataset is simple and does not need preprocessing.
- The Zomato dataset is a real-world dataset, so preprocessing is required before applying machine learning algorithms.
- Both datasets can be used for classification, but the Zomato dataset requires more preparation than the Iris dataset.